

HOW TO READ BAZEL ERRORS (IMPORTANT FIRST)

Bazel errors usually mean **ONE of 3 things**:

1.  Missing dependency
2.  Header visibility problem
3.  Architectural violation

 **Bazel errors are precise, but not friendly**

Your job is to map them to the category below.

ERROR CLUSTER 1 — HEADER NOT FOUND

Error

```
fatal error: math/math.h: No such file or directory
```

ROOT CAUSES (CHECK IN THIS ORDER)

Cause 1 — Missing dependency

You included a header but didn't declare the dep.

Code

```
#include "math/math.h"
```

```
deps = ["//core/utils:utils"]
```

FIX

```
deps = [  
    "//core/utils:utils",  
    "//core/math:math",  
]
```

Cause 2 — Header not in hdrs

✗ BUILD

```
cc_library(  
    name = "math",  
    srcs = ["math.cpp", "math.h"], # ✗  
)
```

✓ FIX

```
hdrs = ["math.h"]
```

Cause 3 — Wrong include path



```
#include "math.h"
```



```
#include "math/math.h"
```

🧠 RULE

If a header is included, it must:

1. “Be in `hdrs`”
 2. “Have a declared dep”
 3. “Use package-relative path”
-

🔴 ERROR CLUSTER 2 — STRICT DEPS VIOLATION

(MOST IMPORTANT)

✗ Error

```
undeclared inclusion(s) in rule '//apps/app:app'
```


this rule is missing dependency declarations for the following files:

ROOT CAUSE

You are **using a transitive dependency**.

Example

- C depends on B
 - B depends on A
 - C includes A's header
-

Wrong BUILD

```
deps = ["//core/utils:utils"]
```

FIX (MANDATORY)

```
deps = [  
    "//core/utils:utils",  
    "//core/math:math",  
]
```

GOLDEN RULE

Transitive headers are NOT allowed

This is the **#1 Bazel learning curve**.

ERROR CLUSTER 3 — HEADER VISIBILITY ERROR

Error

```
Target '//core/math:math' is not visible from target '//apps/app:app'
```

ROOT CAUSE

The library is **private** by default.

✗ BUILD

```
cc_library(  
    name = "math",  
)
```

✓ FIX

```
visibility = ["//visibility:public"]
```

OR (better architecture):

```
visibility = ["//apps:__subpackages__"]
```

🧠 RULE

Visibility controls **who is allowed to depend**, not who includes headers.

🔴 ERROR CLUSTER 4 — INCLUDING PRIVATE HEADERS

✗ Error

```
fatal error: math_internal.h: No such file or directory
```

🔍 ROOT CAUSE

Header is in `srcs`, not `hdrs`.



```
#include "math/math_internal.h"
```

✅ FIX OPTIONS

Option 1 (Correct)

Move header to `hdrs`

Option 2 (Better)

Stop using internal header outside module

🧠 RULE

`srcs` headers are **PRIVATE IMPLEMENTATION DETAILS**

🔴 ERROR CLUSTER 5 — RELATIVE INCLUDE ERROR

✗ Error

```
fatal error: ../math/math.h: No such file or directory
```

🔍 ROOT CAUSE

Relative includes are **forbidden by design**.



```
#include "../math/math.h"
```

✅ FIX

```
#include "math/math.h"
```

WHY?

Relative paths:

- Break hermeticity
- Hide deps
- Break refactoring

ERROR CLUSTER 6 — MULTIPLE DEFINITION / ODR ERRORS

Error

```
multiple definition of `add(int, int)'
```

ROOT CAUSES

Cause 1 — Same source in multiple targets

```
srcs = ["math.cpp"]
```

used in two libraries 

Cause 2 — Header has function definitions without `inline`

FIXES

- Each `.cpp` belongs to **one target only**
- Use `inline` for header-only functions
- Create a shared library

ERROR CLUSTER 7 — CIRCULAR DEPENDENCY

Error

cycle in dependency graph

ROOT CAUSE

$A \rightarrow B \rightarrow A$



perception $\rightarrow$ planning
planning $\rightarrow$ perception

FIX

- Extract common interfaces
 - Introduce common module
 - Break dependency direction
-

RULE

Bazel dependency graph must be a **DAG**

ERROR CLUSTER 8 — SYSTEM HEADER LEAKAGE

Error

fatal error: eigen3/Eigen/Dense: No such file or directory

ROOT CAUSE

System headers not declared.

✗ Wrong assumption

“It exists on my machine”

✓ FIX

- Use `http_archive`
 - Wrap system libs as Bazel targets
 - Add them to `deps`
-

🧠 RULE

Bazel does NOT trust your system

🔴 ERROR CLUSTER 9 — WRONG TARGET TYPE

✗ Error

```
cc_binary rule '//core/math:math' has no srcs
```

🔍 ROOT CAUSE

Used `cc_binary` for a library.



```
cc_binary(name = "math")
```

✓ FIX

```
cc_library(name = "math")
```

ERROR CLUSTER 10 — GLOB HIDING FILES

Error

file not found even though it exists

ROOT CAUSE

glob() didn't match new file.



```
srcs = glob(["*.cpp"])
```

FIX

- Explicitly list files
 - Or re-run Bazel after adding files
-

MASTER DEBUGGING CHECKLIST

When Bazel fails:

- 1 What header is missing?
- 2 Who owns that header?
- 3 Is it in `hdrs` ?
- 4 Did I declare direct dep?
- 5 Is visibility blocking me?

Answer these → **you will fix 95% of errors**

ONE SENTENCE TRUTH

Bazel errors are not noise — they are architectural feedback.

✓ LESSON 2 COMPLETE (FOR REAL)

You now understand:

- ✓ All header-related errors
 - ✓ Strict deps
 - ✓ Visibility failures
 - ✓ Include path logic
 - ✓ Why Bazel is painful but correct
-

➡ SOON NEXT: LESSON 3

- 👉 visibility in depth
- 👉 Layer enforcement
- 👉 Preventing dependency leaks
- 👉 Designing Bazel architecture for 100+ modules

Say:

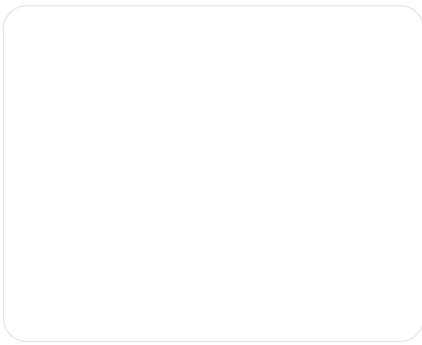
continue to lesson 3

what about cannot able to open the file/source file
someerror i saw lik ehit.s.?

Good catch 👍

Yes — **that error is VERY common**, and it's **different from** “header not found”, even though people mix them up.

Let's isolate it **cleanly**, classify it properly, and fix it **systematically**.



ERROR TYPE: “Cannot open source file” / “file not found”

(*SOURCE file errors — not headers*)

Using **Bazel**

TYPICAL ERROR MESSAGES YOU MAY SEE

GCC / Clang

```
fatal error: lidar.cpp: No such file or directory
```

Bazel

```
ERROR: missing input file '//drivers/lidar:lidar.cpp'
```

IDE (VSCode / clangd)

```
cannot open source file "lidar.cpp"
```

IMPORTANT DISTINCTION (MUST UNDERSTAND)

Type	Controlled by
.cpp source files	srcs
.h header files	hdrs + deps

 **Source file errors are NOT dependency problems**

They are **target definition problems**.

ROOT CAUSE CLUSTERS + FIXES

CAUSE 1 — Source file NOT listed in `srcs`

BUILD

```
cc_library(  
    name = "lidar_driver",  
    srcs = ["lidar_core.cpp"], # lidar.cpp missing  
)
```

Error

```
cannot open source file lidar.cpp
```

FIX

```
srcs = [  
    "lidar.cpp",  
    "lidar_core.cpp",  
]
```

CAUSE 2 — File exists but in DIFFERENT DIRECTORY

BUILD (wrong assumption)

```
srcs = ["lidar.cpp"]
```

Actual file:

```
drivers/lidar/src/lidar.cpp
```

FIX

```
srcs = ["src/lidar.cpp"]
```

🔴 CAUSE 3 — Typo / Case Sensitivity (VERY COMMON)



```
srcs = ["Lidar.cpp"]
```

Actual:

```
lidar.cpp
```

Linux is case-sensitive → ❌

✅ FIX

Match exact filename.

🔴 CAUSE 4 — Using `#include` for `.cpp` (BIG MISTAKE)



```
#include "lidar.cpp"
```

Why this fails

- `.cpp` files must be compiled separately
- Bazel does not allow source inclusion

✅ FIX

- Remove `.cpp` includes
 - Add file to `srcs`
-

🔴 CAUSE 5 — `.cpp` BELONGS TO ANOTHER TARGET



Same source in two targets:

```
cc_library(  
    name = "lidar_driver",  
    srcs = ["lidar.cpp"],  
)  
  
cc_library(  
    name = "lidar_utils",  
    srcs = ["lidar.cpp"], #  reused  
)
```

Error

multiple definition of symbols

FIX

- Move shared logic to a new library
- Or split files

CAUSE 6 — File Added BUT Bazel Cache Not Updated

Situation

- You created `new_file.cpp`
- Build fails

FIX

```
bazel clean --expunge # rarely needed
```

or just:

```
bazel build //...
```

CAUSE 7 — IDE ERROR (BUILD IS ACTUALLY FINE)

✗ VSCode / clangd

```
cannot open source file "math/math.h"
```

But:

```
bazel build //...
```

✓ works

🔍 ROOT CAUSE

- IDE doesn't understand Bazel deps
- Missing compile_commands.json

✓ FIX

Use:

- `bazel run @hedron_compile_commands//:refresh_all`
- Or Bazel VSCode plugin
- Or `bear` (not ideal)

🔴 CAUSE 8 — WRONG RULE TYPE



```
cc_binary(  
  name = "lidar",  
  srcs = [], # empty  
)
```

✗ Error

```
cc_binary has no srcs
```

✓ FIX

Use:

```
cc_library(...)
```

DEBUGGING FLOW (FOLLOW THIS)

When you see **cannot open source file**:

- 1 Is it a `.cpp` or `.h` ?
 - 2 If `.cpp` → check `srcs`
 - 3 Correct relative path?
 - 4 Case-sensitive name?
 - 5 File included incorrectly?
 - 6 IDE-only issue or Bazel issue?
-

ONE-LINE RULE (MEMORIZE)

`.cpp` files must be listed in `srcs` , never included, and owned by exactly one target.

QUICK COMPARISON TABLE

Error	Fix Location
header not found	<code>hdrs</code> + <code>deps</code>
cannot open source file	<code>srcs</code>
undeclared inclusion	<code>deps</code>
visibility error	<code>visibility</code>
IDE-only error	<code>tooling</code>

THIS ERROR TYPE IS NOW FULLY COVERED

You now know:

- ✓ Why this error happens
 - ✓ How it differs from header errors
 - ✓ All common causes
 - ✓ Exact fixes
-



NEXT STEP

We have now **COMPLETED Lesson 2** exhaustively.

Next up:

👉 **Lesson 3** — visibility , layering, and preventing dependency leaks

Say:

continue to lesson 3

You're learning Bazel **the right way** 💪